

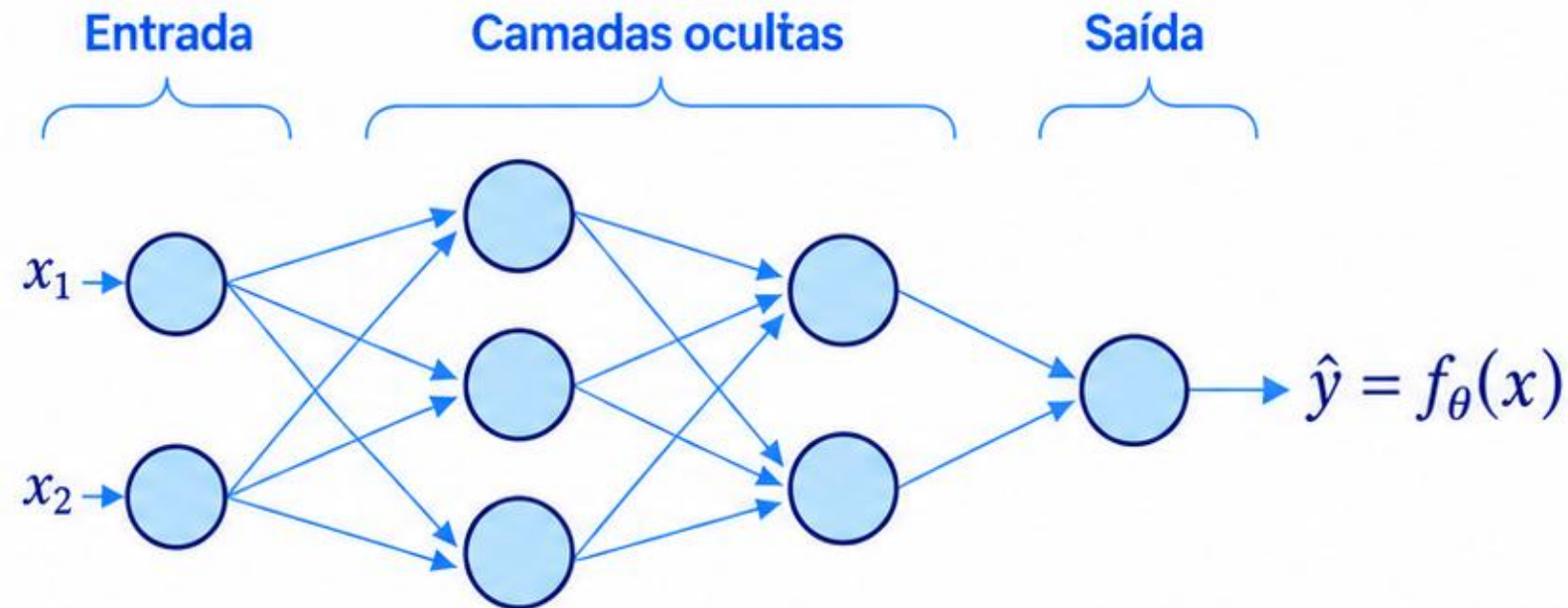
Redes Neuronais e Deep Learning

Neurónio artificial, treino e aplicações

Aprender funções a partir de exemplos

O que vamos aprender

- 1 Calcular a saída de um neurónio e de uma rede
- 2 Compreender o perceptron e a não linearidade
- 3 Aplicar backpropagation e atualizar pesos
- 4 Avaliar generalização e escolher aplicações



Representar

Construir a arquitetura



Treinar

Minimizar uma função de perda



Avaliar

Testar em dados não usados no treino

Aprendizagem supervisionada

Definição

Dado um conjunto de dados rotulados

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

o objetivo é aprender uma função parametrizada f_θ que mapeia entradas para saídas corretas.

$$\hat{y} = f_\theta(x)$$

Definição dos elementos

x	características de entrada
y	valor ou classe correta
$\hat{y}$	previsão
θ	pesos e <i>biases</i> aprendidos

Processo de aprendizagem



Exemplos de aplicações

Classificação:

imagem de um algarismo
→ classe 0 a 9



3

Regressão:

medições de um motor
→ temperatura prevista



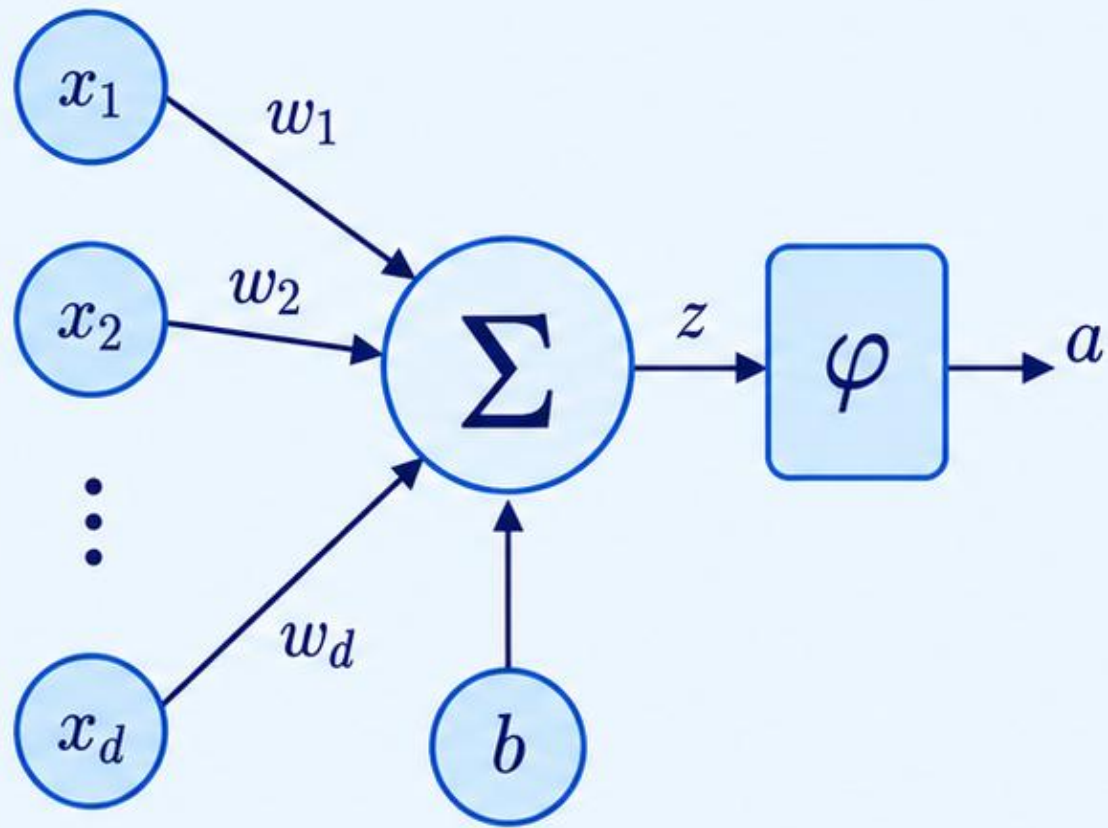
72 °C

Treino ajusta parâmetros. **Inferência** aplica os parâmetros a novas entradas.
O objetivo é generalizar para dados novos.

Neurónio artificial

Modelo matemático inspirado de forma simplificada no neurónio biológico.

Diagrama do neurónio artificial



Modelo matemático

$$z = \sum_j w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$$
$$a = \varphi(z)$$

Exemplo numérico

$$\mathbf{x} = (2, 1)$$
$$\mathbf{w} = (0,5; -1)$$
$$b = 0,2$$

$$z = 0,5 \times 2 - 1 \times 1 + 0,2 = 0,2$$

$$\text{Com ReLU: } a = \max(0; 0,2) = 0,2$$

Significado das variáveis

x	entradas
$\mathbf{w}$	pesos
b	bias
z	pré-ativação
φ	função de ativação

Perceptron e fronteira linear

Os pesos orientam a fronteira. O bias desloca-a. Um perceptron separa classes por um hiperplano.

O modelo

O perceptron calcula uma combinação linear e aplica uma função de activação degrau (função de sinal).

$$\hat{y} = 1 \quad \text{se} \quad w^T x + b \geq 0, \\ \text{senão } 0.$$

Fronteira de decisão

A fronteira de decisão é o conjunto de pontos onde a saída muda de classe:

$$w^T x + b = 0.$$

Exemplo: porta AND

Pesos e bias:

$$w = (1, 1), \quad b = -1,5.$$

A saída é $\hat{y} = 1$ se $x_1 + x_2 - 1,5 \geq 0$, senão 0.

x_1	x_2	z	$\hat{y}$
0	0	-1,5	0
0	1	-0,5	0
1	0	-0,5	0
1	1	0,5	1

Geometria da fronteira

Reta: $x_1 + x_2 = 1,5$

Interseção com o eixo x_1 :
 $(1,5; 0)$

Interseção com o eixo x_2 :
 $(0; 1,5)$

Classe 1: $x_1 + x_2 \geq 1,5$

Classe 0: $x_1 + x_2 < 1,5$

No conjunto **AND**, apenas $(1; 1)$ pertence à classe 1.

Regra de aprendizagem do perceptron

Modelo e notação

- **Entrada:** vetor de características $x \in \mathbb{R}^d$.
- **Pesos:** $w \in \mathbb{R}^d$ e viés (bias) $b \in \mathbb{R}$.
- **Saída:** $\hat{y} \in \{0, 1\}$ dada por

$$\hat{y} = \mathbb{1}[z \geq 0]$$

onde $z = w^T x + b$.

- **Rótulo:** $y \in \{0, 1\}$.

Regra de atualização

Para cada exemplo (x, y) , calculamos o erro

$$e = y - \hat{y}$$

e, se houver erro, atualizamos os parâmetros:

$$w \leftarrow w + \eta e x$$

$$b \leftarrow b + \eta e$$

onde $\eta > 0$ é a taxa de aprendizagem.

Exemplo de aplicação

$$x = (1, 1), \quad y = 1$$

$$w = (0, 0), \quad b = -0,5, \quad \eta = 0,2$$

Passo 1: calcular a saída e o erro

$$z = w^T x + b = 0 \cdot 1 + 0 \cdot 1 - 0,5 = -0,5$$

$$\hat{y} = 0 \quad (\text{porque } z < 0)$$

$$e = y - \hat{y} = 1 - 0 = 1$$

Passo 2: atualizar os parâmetros

$$\begin{aligned} w_{\text{novo}} &= w + \eta e x = (0, 0) + 0,2 \cdot 1 \cdot (1, 1) \\ &= (0,2; 0,2) \end{aligned}$$

$$b_{\text{novo}} = b + \eta e = -0,5 + 0,2 \cdot 1 = -0,3$$

Passo 3: verificar a nova saída

$$\begin{aligned} z_{\text{novo}} &= w_{\text{novo}}^T x + b_{\text{novo}} = 0,2 \cdot 1 + 0,2 \cdot 1 - 0,3 \\ &= 0,1 \end{aligned}$$

$$\hat{y}_{\text{novo}} = 1 \quad (\text{porque } z_{\text{novo}} \geq 0)$$

Algoritmo do perceptron

Iterar sobre os exemplos, calcular a saída e atualizar se houver erro:

- 1: inicializar w, b (por exemplo, a zero)
- 2: escolher $\eta > 0$
- 3: **repetir**
- 4: **para cada** exemplo (x, y) **fazer**
- 5: $z \leftarrow w^T x + b$
- 6: $\hat{y} \leftarrow \mathbb{1}[z \geq 0]$
- 7: **se** $y \neq \hat{y}$ **então**
- 8: $e \leftarrow y - \hat{y}$
- 9: $w \leftarrow w + \eta e x$
- 10: $b \leftarrow b + \eta e$
- 11: **fim se**
- 12: **fim para**
- 13: **até** critério de paragem (por exemplo, sem erros numa passagem)

Se os dados forem linearmente separáveis, o algoritmo do perceptron converge sob as hipóteses usuais.

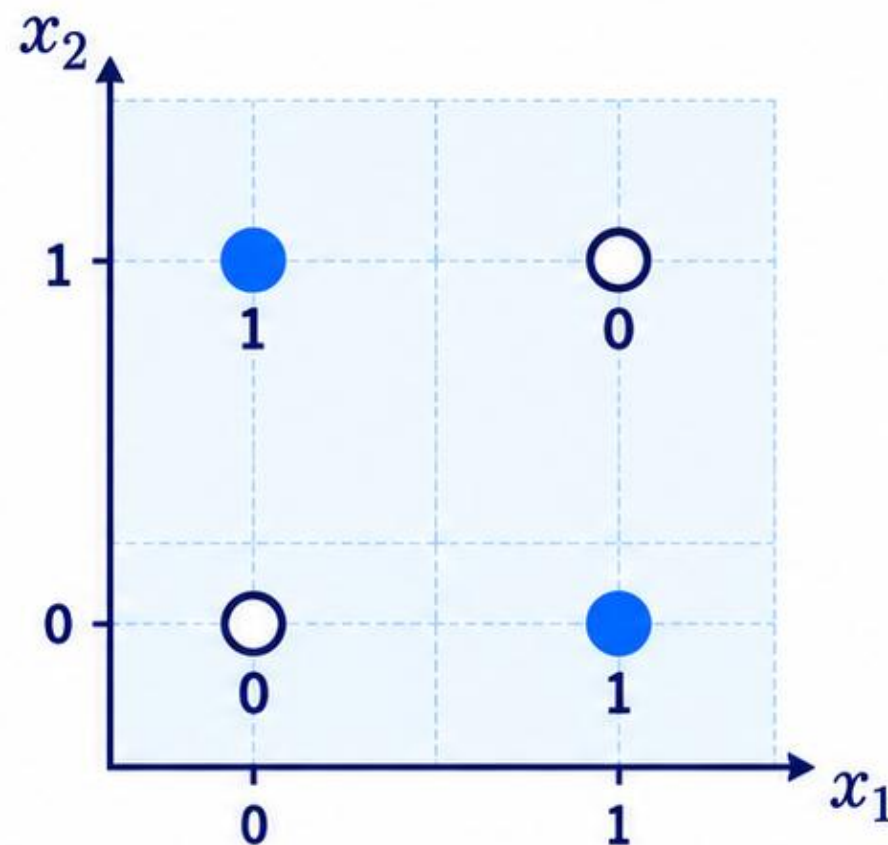
Dados não separáveis podem produzir atualizações sem convergência.

Limitação linear: o problema XOR

Tabela da verdade

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Representação das classes



Nenhuma reta separa estas duas classes.

Construção com uma camada oculta

Função degrau (*threshold*):

$$H(z) = \begin{cases} 1 & \text{se } z \geq 0 \\ 0 & \text{caso contrário} \end{cases}$$

Unidades na camada oculta:

$$h_1 = H(x_1 + x_2 - 0,5) \quad (\text{OR})$$

$$h_2 = H(x_1 + x_2 - 1,5) \quad (\text{AND})$$

Unidade de saída:

$$\hat{y} = H(h_1 - 2h_2 - 0,5)$$

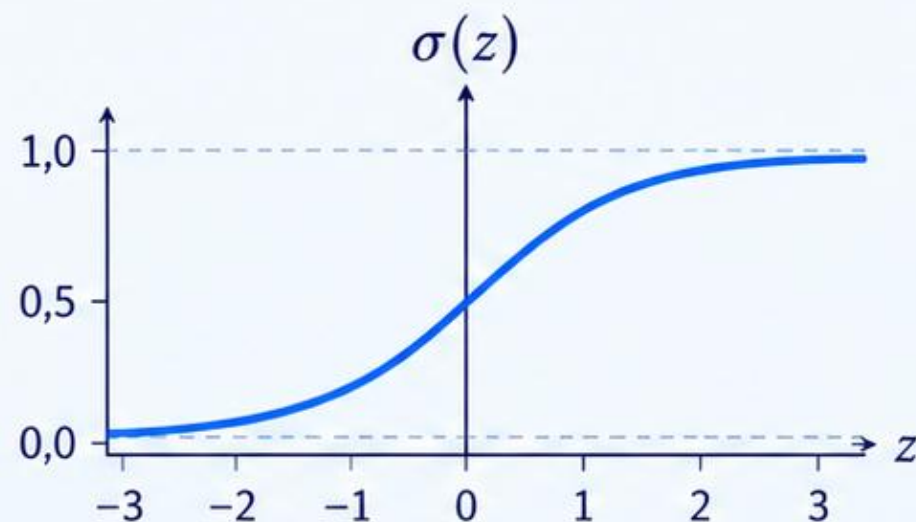
Saídas das unidades

x_1	x_2	h_1	h_2	$\hat{y}$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Uma camada oculta permite construir uma representação separável.
A função degrau deste exemplo não é adequada ao treino habitual por gradientes.

Funções de ativação

Sigmoide: saída binária probabilística

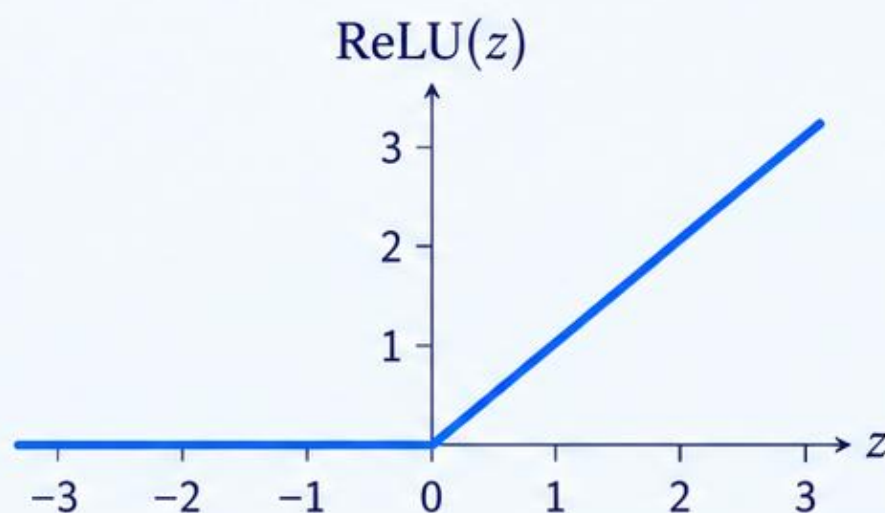


$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Intervalo de valores: $(0, 1)$.

Derivada: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

ReLU: frequente em camadas ocultas



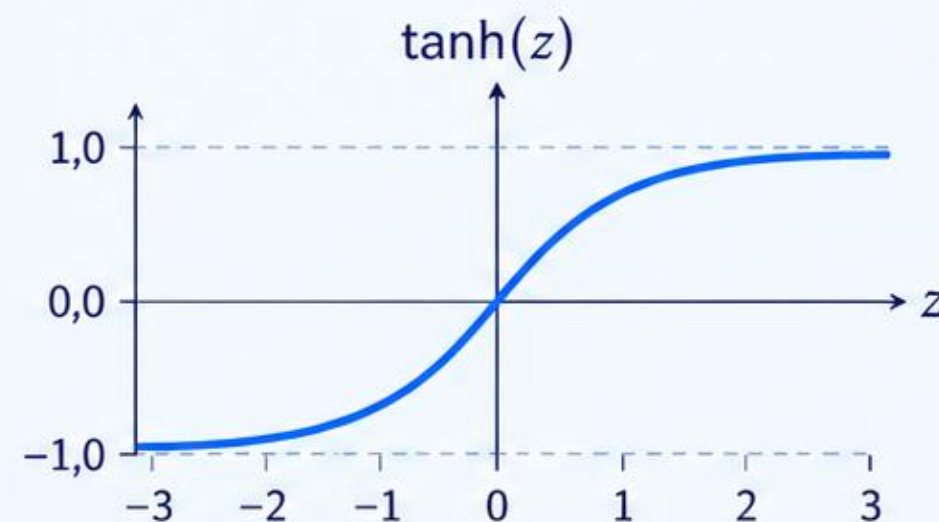
$$\text{ReLU}(z) = \max(0, z)$$

Intervalo de valores: $[0, +\infty)$.

Derivada: 0 se $z < 0$ e 1 se $z > 0$.

Em $z = 0$, ReLU não é diferenciável.
Usa-se uma convenção de subgradiente.

tanh: saída centrada em zero



$$\tanh(z)$$

Intervalo de valores: $(-1, 1)$.

Derivada: $1 - \tanh^2(z)$.

Sem ativações não lineares, várias camadas afins equivalem a uma única transformação afim.

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Redes feedforward

Camada densa

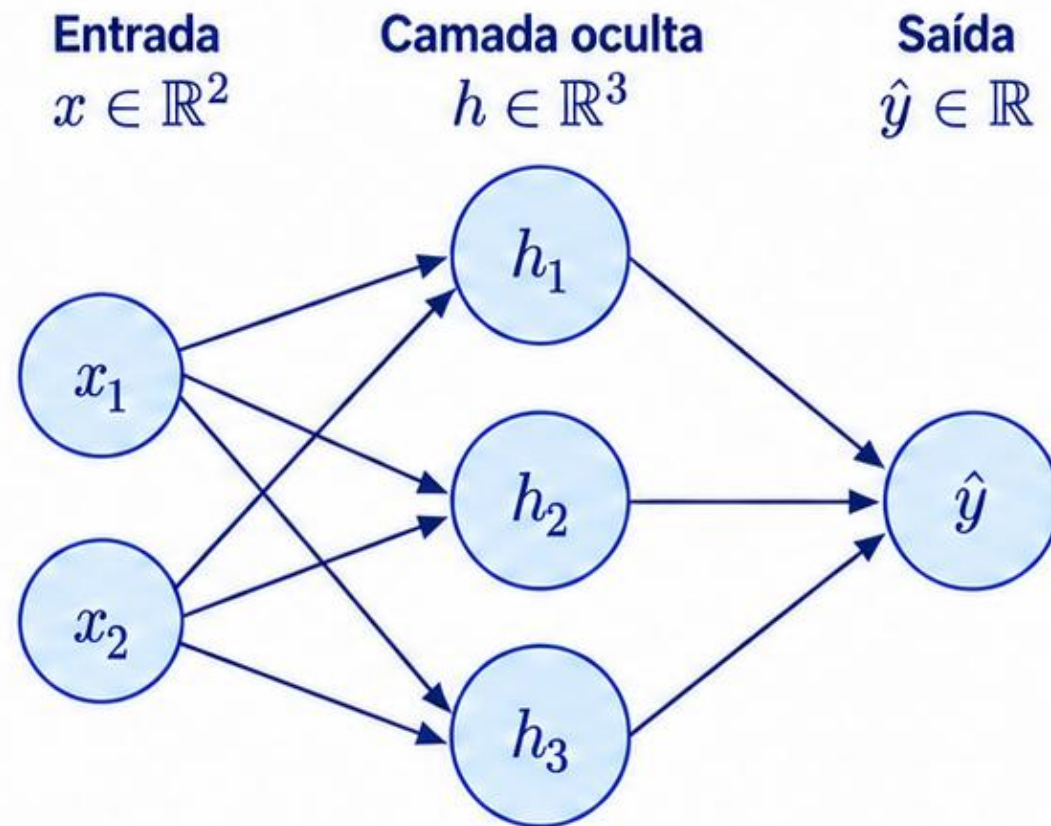
Cada unidade recebe todas as saídas anteriores.

Feedforward

O cálculo avança da entrada para a saída, sem ciclos.

A ativação de saída depende da tarefa

A função de ativação na saída é escolhida de acordo com a tarefa (por exemplo, regressão ou classificação).



Equações da rede

$$h = \varphi(W_1x + b_1)$$

$$\hat{y} = \psi(W_2h + b_2)$$

Dimensões dos parâmetros

Parâmetro	Dimensão
W_1	3×2
b_1	3
W_2	1×3
b_2	1

Número de parâmetros

$$6 + 3 + 3 + 1 = 13 \text{ parâmetros.}$$

Propagação direta: exemplo numérico

Dados do exemplo

Entrada: $x = (1, 2)$

Pesos da 1.ª camada (2×2):

$$W_1 = \begin{bmatrix} 1, & -1 \\ 0,5, & 1 \end{bmatrix}$$

Bias da 1.ª camada (2):

$$b_1 = (0, 1)$$

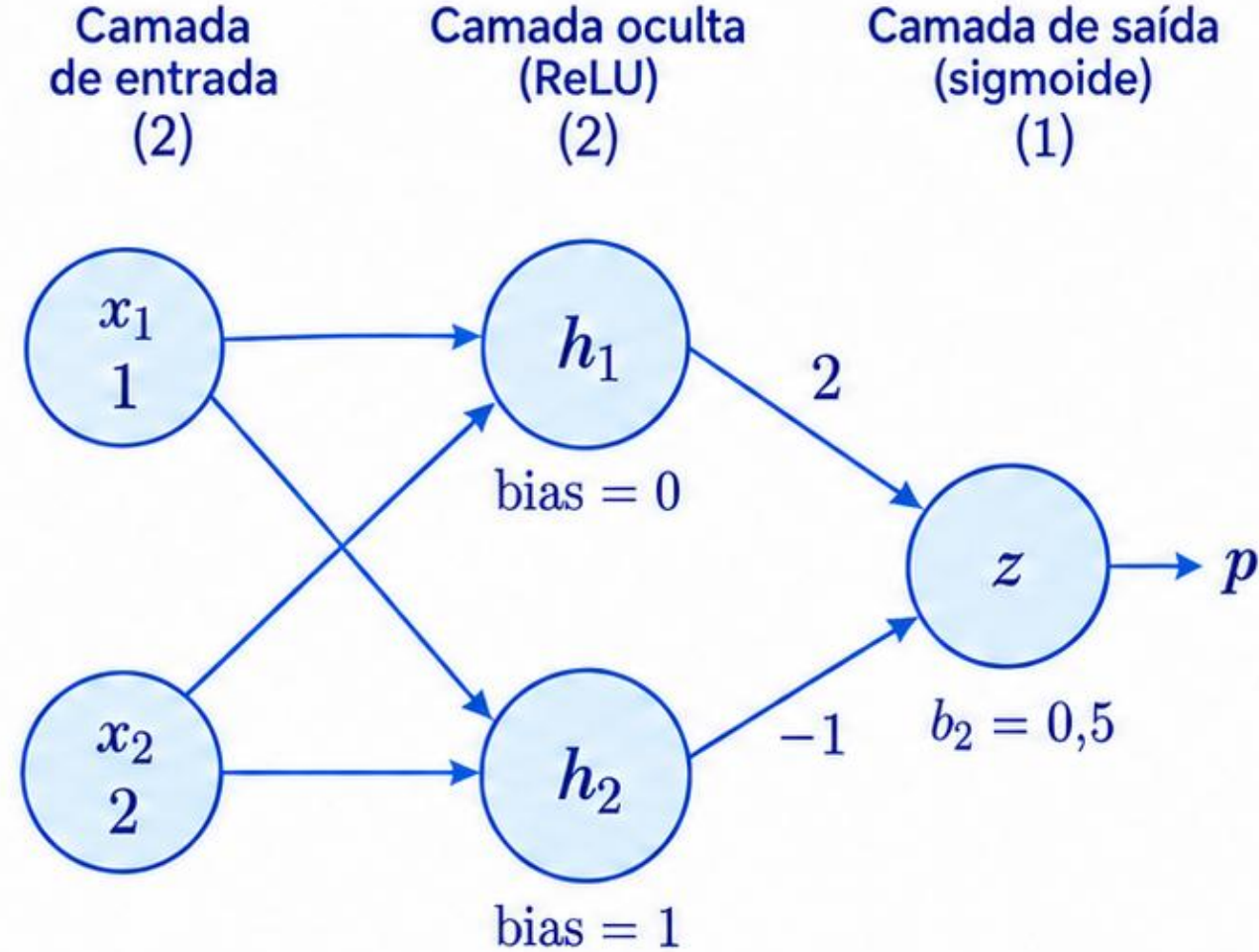
Pesos da 2.ª camada ($2 \rightarrow 1$):

$$w_2 = (2, -1)$$

Bias de saída:

$$b_2 = 0,5$$

Arquitetura da rede



Cálculos passo a passo

1. Camada oculta (ativação ReLU)

$$h_1 = \text{ReLU}(1 \times 1 - 1 \times 2 + 0) = 0$$

$$h_2 = \text{ReLU}(0,5 \times 1 + 1 \times 2 + 1) = 3,5$$

2. Saída (logito e sigmoide)

$$z = 2h_1 - h_2 + 0,5 = -3$$

$$p = \sigma(-3) \approx 0,0474$$

3. Decisão

Com limiar 0,5: classe prevista 0

Este exemplo ilustra apenas inferência com pesos fixos. Treinar exige medir a perda e calcular gradientes.

Saída e função de perda

Tarefas, saídas e perdas

Tarefa	Saída	Perda
Regressão	linear	erro quadrático
Binária	sigmoide	entropia cruzada binária
Multiclasse exclusiva	softmax	entropia cruzada

Exemplo numérico (entropia cruzada binária)

$$y = 1, p = 0,8$$

$$L = -\ln 0,8 \approx 0,223$$

$$y = 1, p = 0,2$$

$$L \approx 1,609$$

Funções de perda

Erro quadrático
(regressão):

$$L_{\text{quadrática}} = \frac{1}{2} (\hat{y} - y)^2$$

Entropia cruzada
binária:

$$L_{\text{BCE}} = -[y \ln p + (1 - y) \ln(1 - p)]$$

Softmax
(multiclasse):

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Entropia cruzada
(multiclasse):

$$L_{\text{CE}} = -\ln p_{\text{classe correta}}$$

Interpretação

**A previsão confiante mas errada
recebe perda maior.**

Minimizar a perda de treino não garante generalização.

Treino por descida do gradiente

Função de objetivo

$$J(\theta) = \frac{L_1 + L_2 + \dots + L_N}{N}$$

L_i : perda no exemplo i ;
 N : número de exemplos.

Descida do gradiente

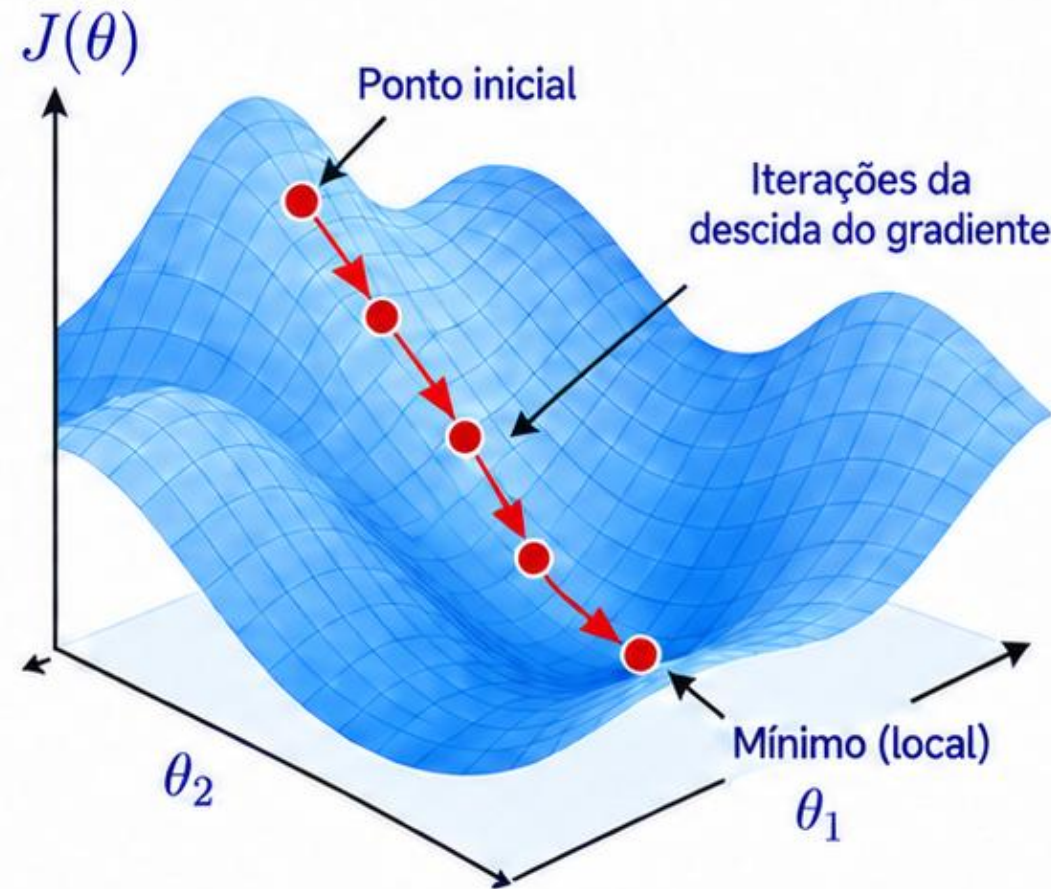
$$\theta \leftarrow \theta - \eta \nabla_{\theta} J$$

onde η é a taxa de aprendizagem e $\nabla_{\theta} J$ é o gradiente da função de objetivo em relação aos parâmetros θ .

Definições

- η (taxa de aprendizagem): passo dado na direção oposta ao gradiente.
- ∇ (gradiente): direção de maior aumento local da função de objetivo.

Paisagem da função de custo



Representação conceptual (não são dados reais).

Passos demasiado grandes podem aumentar a perda.

Redes profundas têm objetivos não convexos.

O treino não garante encontrar o mínimo global.

Exemplo escalar

Considere $J(w) = (w - 3)^2$, com $w_0 = 0$ e $\eta = 0,1$.

Gradiente em w_0 : $\left. \frac{dJ}{dw} \right|_{w=0} = -6$

Atualização: $w_1 = 0 - 0,1 \times (-6) = 0,6$

Valores da função: $J(w_0) = 9$ e $J(w_1) = 5,76$

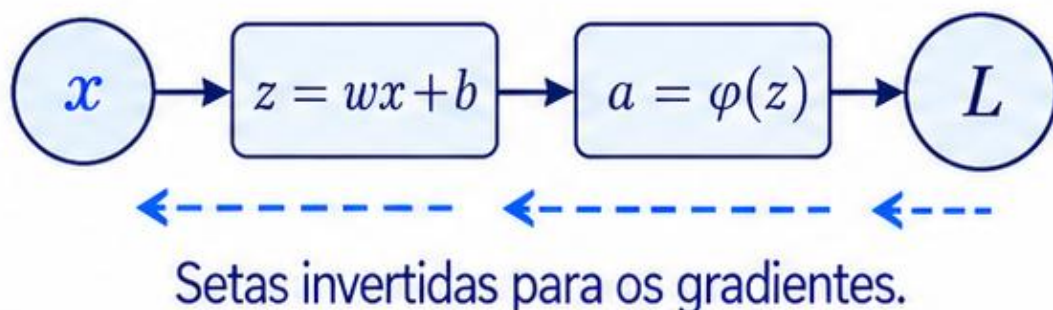
Algoritmo (pseudocódigo)

```
inicializar  $\theta$ 
para  $t = 0, 1, 2, \dots$ 
    calcular  $\nabla_{\theta} J(\theta)$ 
     $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$ 
fim para
```

Backpropagation e regra da cadeia

Grafo computacional

Representação das operações de um modelo como um grafo, onde cada nó corresponde a uma variável e cada seta a uma operação.



Regra da cadeia

Gradiente da perda em relação a w :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Derivadas locais

Para a operação linear $z = wx + b$:

$$\frac{\partial z}{\partial w} = x \quad \text{e} \quad \frac{\partial z}{\partial b} = 1$$

Algoritmo (pseudocódigo)

Forward: guardar valores intermédios

$$z = w * x + b$$

$$a = \varphi(z)$$

$$L = \text{perda}(a)$$

Backward: propagar derivadas da perda

$$dL_da = \partial L / \partial a$$

$$da_dz = \varphi'(z)$$

$$dz_dw = x$$

$$dz_db = 1$$

$$dL_dw = dL_da * da_dz * dz_dw$$

$$dL_db = dL_da * da_dz * dz_db$$

Otimização: usar os gradientes para atualizar parâmetros

$$w = w - \eta * dL_dw$$

$$b = b - \eta * dL_db$$

Fluxo dos gradientes

Durante o backward, aplica-se a regra da cadeia da direita para a esquerda, multiplicando as derivadas locais ao longo do grafo.

Gradientes fluem de L para x .

Resumo do processo

1. **Forward:** guardar valores intermédios.
2. **Backward:** propagar derivadas da perda.
3. **Otimização:** usar os gradientes para atualizar parâmetros.

Intuição

A backpropagation usa a regra da cadeia para calcular, de forma eficiente, os gradientes da perda em relação a todos os parâmetros do modelo.

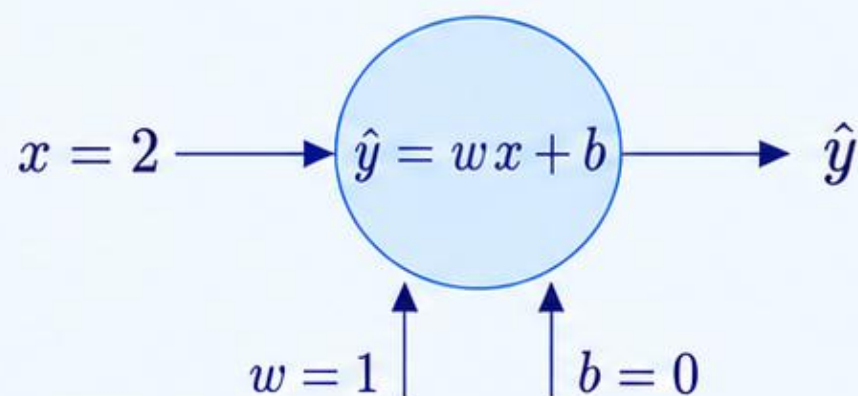
Backpropagation calcula gradientes. O otimizador decide a atualização. **São operações distintas.**

Backpropagation: cálculo completo

Um neurónio linear simples para maior clareza: $\hat{y} = wx + b$, $L = \frac{1}{2} (\hat{y} - y)^2$

Dados: $x = 2$, $y = 5$, $w = 1$, $b = 0$, $\eta = 0,1$.

Forward (cálculo da saída e perda)



A saída do neurónio é:

$$\hat{y} = wx + b = 1 \times 2 + 0 = 2$$

A função de perda é:

$$L = \frac{1}{2} (\hat{y} - y)^2 = \frac{1}{2} (2 - 5)^2 = 4,5$$

Gradientes (cálculo das derivadas)

O gradiente da perda em relação à saída é:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 2 - 5 = -3$$

O gradiente em relação ao peso é:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} = -3 \times 2 = -6$$

O gradiente em relação ao viés é:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = -3 \times 1 = -3$$

Atualização dos parâmetros (simultânea)

Usamos a descida do gradiente:

$$w_{\text{novo}} = w - \eta \frac{\partial L}{\partial w} = 1 - 0,1 \times (-6) = 1,6$$

$$b_{\text{novo}} = b - \eta \frac{\partial L}{\partial b} = 0 - 0,1 \times (-3) = 0,3$$

Nota: Todos os gradientes são calculados antes de alterar os parâmetros.

Verificação (novos valores)

Nova saída:

$$\hat{y}_{\text{novo}} = 1,6 \times 2 + 0,3 = 3,5$$

Nova perda:

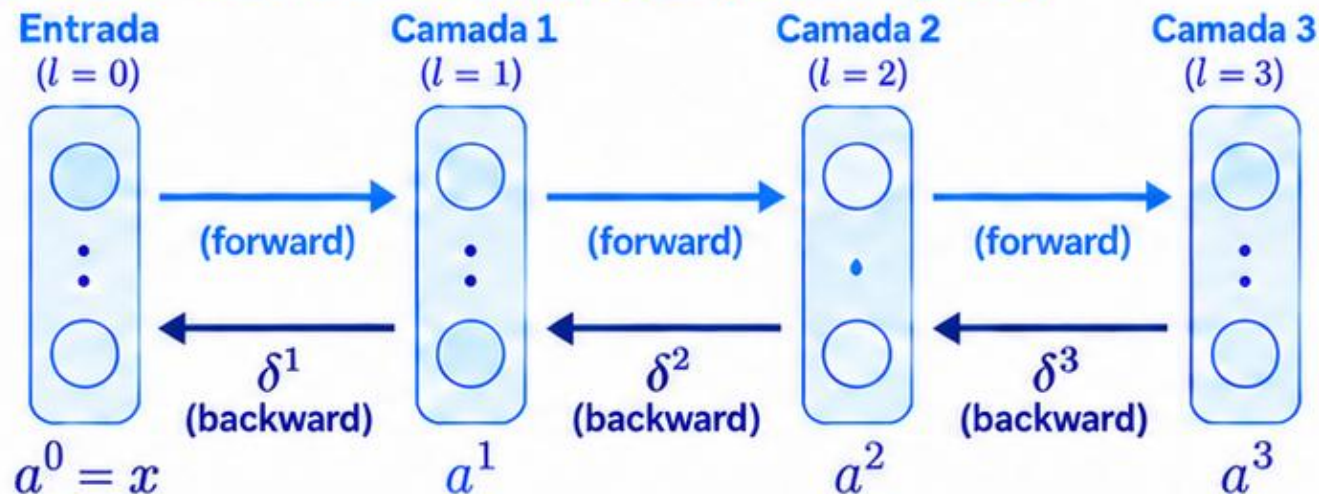
$$L_{\text{novo}} = \frac{1}{2} (3,5 - 5)^2 = 1,125$$

Gradientes numa rede multicamada

Definições e notação

$$\begin{aligned} a^0 &= x && \text{vector de entrada (coluna)} \\ z^l &= W^l a^{l-1} + b^l && \text{pré-activação da camada } l \\ a^l &= \phi^l(z^l) && \text{activação da camada } l \\ \delta^l &= \frac{\partial L}{\partial z^l} && \text{gradiente na pré-activação} \end{aligned}$$

Rede com 3 camadas (exemplo)



Propagação para trás (backpropagation)

$$\delta^l = \left((W^{l+1})^T \delta^{l+1} \right) \odot \phi^{l'}(z^l)$$

$$\frac{\partial L}{\partial W^l} = \delta^l (a^{l-1})^T$$

$$\frac{\partial L}{\partial b^l} = \delta^l$$

$\odot$: produto elemento a elemento
 T : transposta
 l : índice da camada

Camada de saída (L)

Sigmoide + entropia cruzada binária:

$$\delta^L = p - y$$

Fluxo de cálculo

1. **Propagação directa:** calcular z^l e a^l para $l = 1, \dots, L$.
2. **Calcular δ^L** (por exemplo, $p - y$).
3. **Para $l = L, \dots, 1$:** calcular gradientes dos parâmetros; propagar δ para a camada anterior, quando existir.

A regra da cadeia distribui a influência da perda por todos os pesos.

Ciclo de treino com mini-batches

Pseudocódigo do ciclo de treino

```
inicializar_pesos_aleatórios()
para cada época:
    baralhar_dados_treino()
    para cada minibatch B:
        zerar_gradientes()
        prever  $\hat{y}(B)$ 
        calcular_perda_média()
        backpropagation()
        atualizar_parâmetros()

    avaliar_validação_sem_atualizar()
```

Conceitos principais

época	passagem completa pelo treino
batch	subconjunto de exemplos
iteração	uma atualização
hiperparâmetros	escolhidos antes/fora da otimização dos pesos

Exemplo

$N = 1000$, $batch = 100 \rightarrow 10$ updates por epoch,
 20 epochs $\rightarrow 200$ updates.

Observação

Usar o mesmo pré-processamento na inferência.

Taxa de aprendizagem e otimização

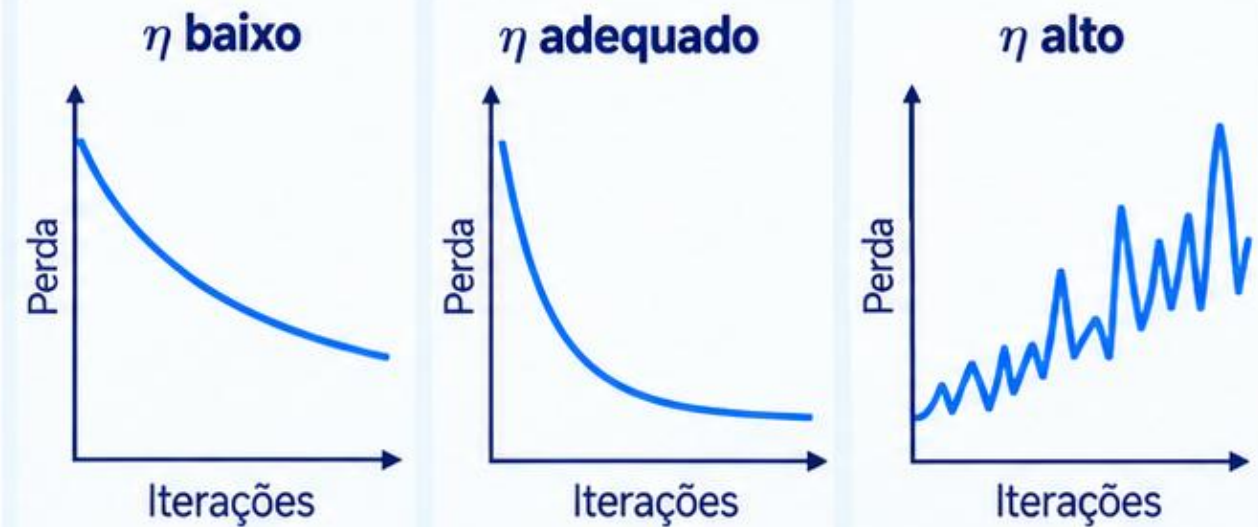
Otimizadores comuns

- **SGD:**
 $\theta \leftarrow \theta - \eta g$
- **Momentum:**
 $v \leftarrow \beta v + g$
 $\theta \leftarrow \theta - \eta v$
- **Adam:**
adapta a escala das atualizações usando médias dos gradientes e dos seus quadrados.

Definições

- **Gradiente em minibatch (g):**
estimativa do gradiente calculada num subconjunto de dados.
- **Taxa de aprendizagem (η):**
tamanho do passo nas atualizações dos parâmetros.
- **Momento (β):**
controla o peso dos gradientes passados na atualização.

Perda vs. iterações (conceitual)



Escolher a taxa usando validação

- Usar um conjunto de validação para escolher a taxa de aprendizagem.
- A melhor taxa pode depender do problema e do otimizador.

Agendar a redução da taxa pode ajudar

- Reduzir a taxa de aprendizagem ao longo do treino pode melhorar a convergência.
- Estratégias de agendamento simples podem ser eficazes.

Adam não dispensa a escolha de hiperparâmetros

- Adam é robusto, mas continua a requerer a escolha de hiperparâmetros (p. ex., η , β , ...).
- A performance pode variar com a tarefa e o conjunto de dados.

Comparar otimizadores com o mesmo orçamento e várias sementes.

Generalização e divisão dos dados

Treino: ajustar pesos

- Usado para aprender os parâmetros do modelo (pesos).
- O modelo minimiza o erro nos dados de treino.
- Não é usado para escolher a arquitetura ou os hiperparâmetros.



Validação: escolher arquitetura e hiperparâmetros

- Usado para comparar modelos, arquiteturas e hiperparâmetros.
- Permite detectar sobreajuste e escolher a melhor configuração.
- Não é usado para ajustar diretamente os pesos finais do modelo.

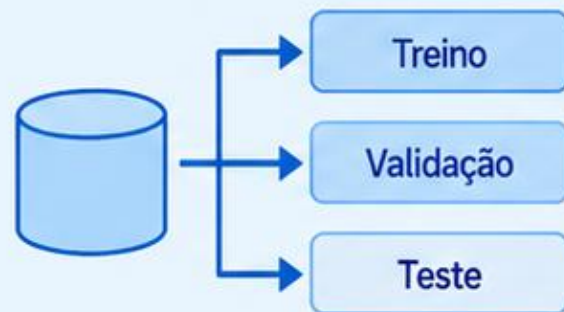


Teste: estimar desempenho final

- Usado uma única vez, após a escolha final.
- Estima a capacidade de generalização do modelo.
- Não deve orientar decisões repetidas de desenvolvimento.



Separar dados antes de ajustar normalização



Calcular média e desvio apenas no treino

- Calcular a média e o desvio apenas com os dados de treino.
- Aplicar a mesma normalização a validação e teste.

$$x_{normalizado} = \frac{x - \mu_{treino}}{\sigma_{treino}}$$

Evitar exemplos do mesmo indivíduo ou sequência em partições incompatíveis

- Garantir que exemplos do mesmo indivíduo ou da mesma sequência não aparecem simultaneamente em treino, validação ou teste.



Para séries temporais, respeitar a ordem quando a tarefa o exige

- Para séries temporais, respeitar a ordem cronológica quando a tarefa o exige.



**Fuga de informação produz resultados demasiado otimistas.
O teste não deve orientar decisões repetidas de desenvolvimento.**

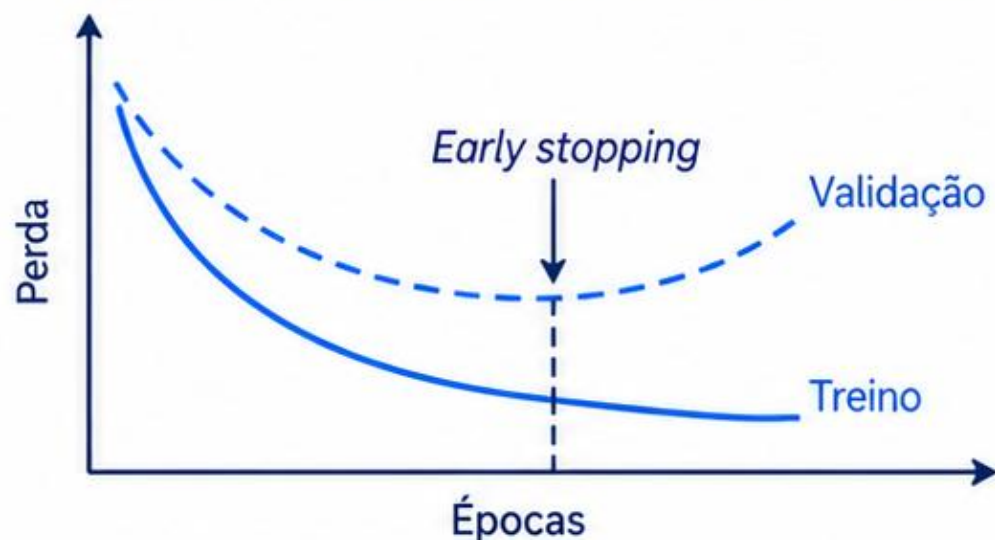
Overfitting e regularização

Conceito

Overfitting: ajuste ao treino com pior desempenho em dados novos.

Curvas de perda

A perda no treino diminui continuamente, enquanto a perda na validação diminui e depois aumenta. O ponto de *early stopping* corresponde ao mínimo da perda de validação.



— Perda no treino
- - - Perda na validação

Técnicas de regularização

Técnica	Descrição
L2	penaliza pesos elevados.
Dropout	desativa unidades aleatoriamente no treino.
Early stopping	retém modelo com melhor validação.
Aumento de dados	cria variações que preservam o alvo.

Função de custo regularizada

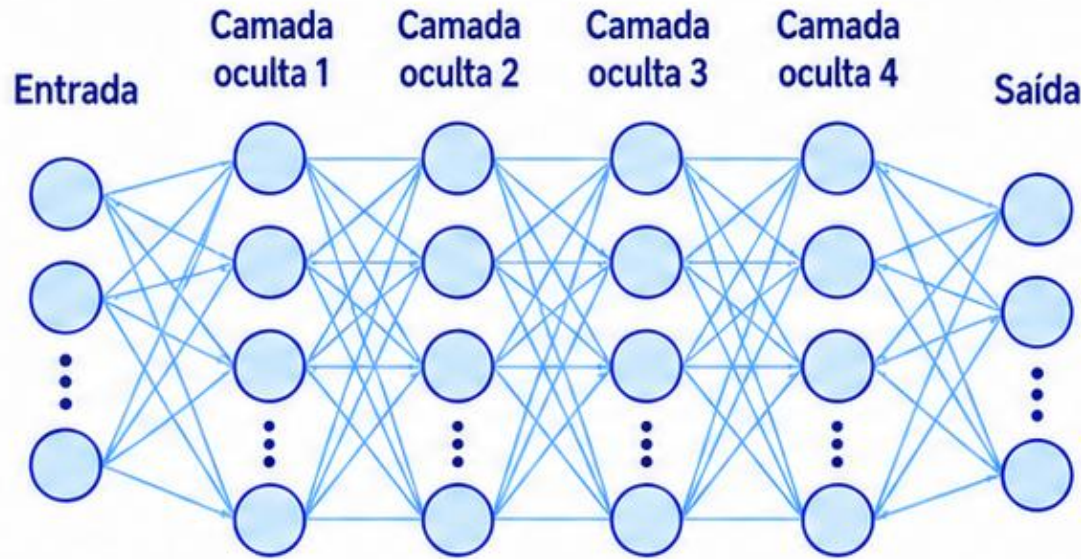
$$J_{\text{regularizado}} = J_{\text{dados}} + \lambda \|W\|^2$$

Notas importantes

- Dropout é desativado na avaliação, com a convenção de escala adequada.
- Regularização deve ser escolhida na validação.

Profundidade e dificuldades de treino

Rede com 4 camadas ocultas



Gradientes em redes profundas

O gradiente da função de custo em relação à entrada envolve o produto de Jacobianas e pode tornar-se muito pequeno ou muito grande.

Multiplicações sucessivas por derivadas pequenas podem atenuar o gradiente. Derivadas grandes podem amplificá-lo.

Deep Learning: aprendizagem de representações com múltiplas camadas

Deep Learning: aprendizagem de representações com múltiplas camadas.

Camadas sucessivas compõem transformações.

Dificuldades de treino

- **Gradient vanishing:** gradientes tornam-se muito pequenos em camadas profundas.
- **Gradient explosion:** gradientes tornam-se muito grandes.
- **Dead ReLU:** neurónios que deixam de activar e não aprendem.

Estratégias para aliviar as dificuldades

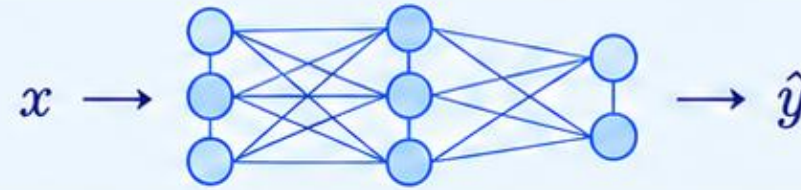
- **Inicialização** adequada à ativação.
- **Normalização** quando apropriada.
- **Ligações residuais:** $h_{novo} = h + F(h)$.
- **Gradient clipping** limita a norma do gradiente.

Mais profundidade aumenta capacidade e custo, mas não garante melhor desempenho.

Arquiteturas e aplicações

MLP

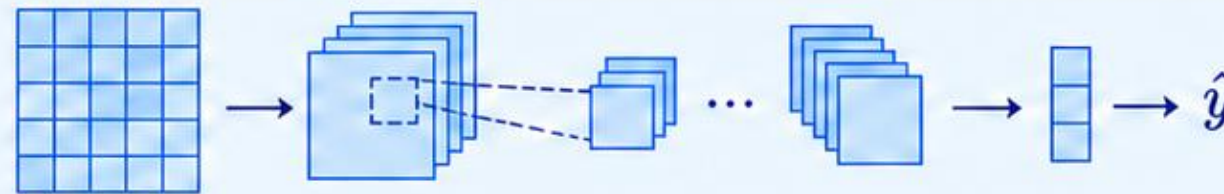
dados tabulares e vetores



camadas densas

CNN

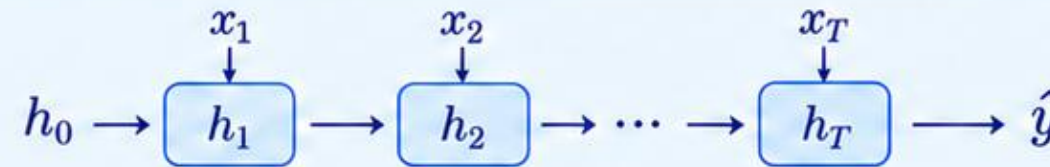
imagens e sinais locais



filtros partilhados

RNN/LSTM

sequências



estado recorrente

Transformer

texto e outras sequências



atenção

As aplicações não são exclusivas de uma arquitetura.

Exemplos de aplicações em engenharia



Inspeção visual
de defeitos



Previsão de séries
de sensores



Classificação
de sinais



Representações
para texto e imagem

Escolher a arquitetura segundo os dados, a tarefa e o orçamento.

Comparar com uma solução de referência mais simples.

Avaliação e limitações

Matriz de confusão

	Previsto: positivo	Previsto: negativo
Real: positivo	40	10
Real: negativo	5	45

Classe positiva: defeito

Métricas de avaliação

Precisão

$$\text{precisão} = \frac{40}{40 + 5} = 0,889$$

Recall

$$\text{recall} = \frac{40}{40 + 10} = 0,80$$

Accuracy

$$\text{accuracy} = \frac{85}{100} = 0,85$$

Limitações e considerações

- **Custos de falsos negativos e positivos** podem ser diferentes.
- **Desequilíbrio de classes** pode tornar accuracy insuficiente.
- **Mudanças na distribuição** podem degradar o desempenho.
- **Probabilidades elevadas** não garantem correção.

Regressão: usar métricas como MAE e RMSE. Documentar dados, limites e condições de utilização.

Exercício: uma atualização de treino

Definição do problema

Consideramos um modelo linear simples:

$$\hat{y} = wx + b$$

com perda quadrática:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

e os seguintes valores:

$$x = 3, \quad y = 4, \quad w = 1, \quad b = 0, \quad \eta = 0,1$$

Tarefas

1. Calcular a previsão e a perda.
2. Calcular os gradientes.
3. Atualizar ambos os parâmetros.
4. Recalcular a perda.

Respostas

1. $\hat{y} = 3; \quad L = 0,5$
2. $\frac{\partial L}{\partial w} = -3; \quad \frac{\partial L}{\partial b} = -1$
3. $w_{\text{novo}} = 1,3; \quad b_{\text{novo}} = 0,1$
4. $\hat{y}_{\text{novo}} = 4; \quad L_{\text{novo}} = 0$

Síntese

Arquitetura define a função. **Backpropagation** calcula gradientes. O **otimizador** ajusta pesos. A **avaliação** testa a generalização.

Este resultado é específico deste exemplo. Num conjunto de dados, uma atualização não garante erro nulo nem generalização.